

Custom: Instructions, Prompts, Skills, Agents with GitHub Copilot



Teaching Copilot Your Team's Standards and Workflows

What Are Custom Instructions?

Document *how* to work with your code, such as coding standards, preferred technologies, or project requirements



Markdown Files

Plain .md files that provide context to GitHub Copilot about your project's conventions, patterns, and requirements



Better Suggestions

Help Copilot generate more relevant code by understanding your tech stack, coding standards, and architectural patterns



Repository or Path-Specific

Can be applied to an entire repository or scoped to specific file paths using glob patterns



Version-Controlled & Shared

Stored in your repo, version-controlled with Git, automatically shared with all team members

Repository-Wide vs Path-Specific

Two scoping levels for custom instructions

Repository-Wide

```
.github/copilot-instructions.md
```

- ✓ Automatically loaded when working in that repo
- ✓ Apply to all files unless overridden
- ✓ Works in VS Code and supported IDEs
- ✓ Single file for entire project standards
- ✓ Architectural decisions and Coding conventions

Path-Specific

```
.github/instructions/*.instructions.md
```

```
# frontend.instructions.md
---
applyTo: "frontend/**/*.js,
        frontend/**/*.jsx"
---
```

- ✓ Scoped by glob patterns (applyTo)
- ✓ frontend.instructions.md for React
- ✓ backend.instructions.md for API
- ✓ Override or complement repo-wide



Which approach should you use?

Start with a single `.github/copilot-instructions.md` file for project-wide coding standards. Add `.instructions.md` files when you need different rules for different file types or frameworks.

Setting Up Custom Instructions

Three levels: Organization, Repository, and VS Code

Organization Level

- 1 GitHub.com → Profile → Organizations
- 2 Settings → Copilot → Custom Instructions
- 3 Add natural language instructions (text box)
- 4 Supported: Chat, Code Review, Coding Agent

Repository Level

- 1 Create .github/copilot-instructions.md
- 2 Add markdown with standards and patterns
- 3 Version-controlled, shared with team
- 4 Works in VS Code and supported IDEs

To Enable / Disable in VS Code

- 1 Cmd+, (Mac) / Ctrl+, (Win/Linux)
- 2 Search: instruction file

[Sample Instruction File](#)

Custom instructions are not visible in Chat — verify via the References list in responses.

Content Guidelines & Limitations

What to include in custom instructions and what to watch out for

What to Include

- ✓ Use markdown formatting for clarity
- ✓ Be specific and actionable (not vague)
- ✓ Coding standards and naming conventions
- ✓ Tech stack and version numbers
- ✓ Architectural patterns and frameworks
- ✓ Error handling and security practices

Limitations to Know

- ✗ Instructions are suggestions, not guarantees
- ✗ Complex rules may be ignored for simple tasks
- ✗ Too many instructions reduce suggestion quality
- ✗ Code Review instructions capped at 4,000 characters
- ✗ Keep repo instructions under ~1,000 lines for best results



Meta-Prompt: Generate Your Instructions File

Role: Senior Developer with enterprise experience.

Task: Create copilot-instructions.md for Python + React project.

Backend and Frontend folders in same workspace. Split Instructions between root and projects.

Divide into sections. Keep under 2000 words. Ask questions before proceeding.

The AGENTS.md Convention

Open standard for agent-readable instructions — lives at repo root

Repo-Root File

Single AGENTS.md at the repository root — the convention adopted across many AI coding agents

Cross-Tool Standard

Works with GitHub Copilot, Claude Code, Cursor, and other tools that honour the AGENTS.md standard

Complements `.github/copilot-instructions.md`

Copilot reads AGENTS.md alongside `.github/copilot-instructions.md` — they layer, they don't conflict

Human-Readable First

Write for humans too — onboarding docs and agent instructions in one place means no drift



Prompt Files

What Are Prompt Files?

Standalone Markdown files that encode reusable prompts as slash commands

 **Prompt files** are Markdown files with the `.prompt.md` extension, stored in `.github/prompts` folder. They have optional YAML frontmatter + Markdown body.

Slash Commands

Type `/` in chat followed by the prompt name to invoke instantly

Markdown Format

`.prompt.md` files with YAML frontmatter for configuration + Markdown body for instructions

Repository Location

Store in `.github/prompts/` folder — shared with your entire team via version control

Configurable

Set agent mode, model, tools, description, and argument hints via YAML frontmatter

Available Variables

Dynamic values you can reference in prompt files

Workspace Variables

`${workspaceFolder}`

Path to the root of your workspace/project

Selection Variables

`${selection}`

Currently selected code in the editor

`${selectedText}`

Text content of the current selection

File Context Variables

`${file}`

Path to the currently active file

`${fileDirname}`

Directory of the currently active file

Input Variables

`${input:variableName}`

Custom input passed from the chat input field

Example: "Review the code in `${file}` focusing on `${input:reviewFocus}`" — makes prompts flexible and reusable

Anatomy of a Prompt File

YAML frontmatter header + Markdown body instructions

YAML Frontmatter (Optional Header)

```
---
agent: 'agent'
model: GPT-5
tools: ['githubRepo', 'codebase']
description: 'Generate a React form'
argument-hint: 'mention form name'
---
```

Frontmatter Properties

Property	Purpose
agent	'agent' (agentic) or 'ask' (Q&A)
model	AI model to use (GPT-5.2, Claude, etc.)
tools	Array of tools/MCP servers to enable
description	Shown in slash command menu
argument-hint	Placeholder text for extra input

Markdown Body (Instructions)

```
Your goal is to generate a new React form component based on templates.
Requirements:
* Use form design system components: [Form.md](../docs/design-system/Form.md)
* Use react-hook-form for state management
* Use yup for validation with TypeScript types
```

Use Cases for Prompt Files

Encode repetitive tasks — invoke with a single slash command

Scaffold Components

Generate new React/Angular/Vue components from templates with consistent structure and naming

Run & Fix Tests

Execute test suites and automatically generate fixes for failing tests with context

Prepare Pull Requests

Auto-generate PR descriptions, changelogs, and review checklists from your changes

Override Agent Behavior

Create minimal implementation plans or generate API mockups with custom agent instructions

Security Reviews

Run standardized security audits with consistent checklist criteria across the team

Generate Documentation

Create API docs, README templates, or architecture decision records from code context

[Custom: GenerateUnitTest Prompt](#)

[Custom CodeReview Prompt](#)

Tips for effective prompts

- ✓ Clearly describe what the prompt should accomplish and the expected output format
- ✓ Provide examples of expected input and output to guide AI responses
- ✓ Use Markdown links to reference custom instructions — don't duplicate guidelines
- ✓ Use `${selection}` and `${input:var}` variables to make prompts flexible and reusable
- ✓ Test prompts with the editor play button and refine based on results



Agent Skills

Custom Agent Skill in GitHub Copilot

What Are Agent Skills?

Specialized folders of instructions, scripts, and resources for Copilot

Agent Skills are folders of instructions, scripts, and resources that Copilot can load **when relevant** to improve its performance in specialized tasks. Agent Skills is an open standard used by a range of different agents.

Example Skill Categories

-  Language & framework expertise
-  Architecture rules
-  Coding standards
-  Security practices
-  Testing strategy
-  Cloud / infrastructure assumptions
-  Performance expectations

Shared Skill Libraries

```
.github/skills/<skill-name>/SKILL.md
```

```
.claude/skills
```

```
.agents/skills/<skill-name>/SKILL.md
```

Creating an Agent Skill

Directory structure, SKILL.md frontmatter, and content guidelines

Directory Structure

```
.github/skills/  
  webapp-testing/  
    SKILL.md           # Instructions  
    test-template.js   # Template  
    examples/          # Examples  
      login-test.js  
      search-test.js
```

SKILL.md Format

```
---  
name: webapp-testing  
description: Guide for testing  
  web apps using Playwright.  
  Use when asked to create or  
  run browser-based tests.  
---  
  
# Web Application Testing  
[Instructions, examples, guidelines]
```

Skill Body Should Include

- What the skill helps accomplish
- When to use the skill (triggers)
- Step-by-step procedures to follow
- Examples of expected input/output
- References to included scripts/resources

[Sample Skill](#) (UI Test using Playwright)

Skills are available as **slash commands** in chat, alongside prompt files

How Copilot Uses Skills

Three-stage process: Discovery → Loading → Resource Access

1

Skill Discovery

Copilot always knows which skills are available by reading their name and description from the YAML frontmatter. This metadata is lightweight and helps Copilot decide relevance.

`Reads: name + description
from YAML frontmatter`

2

Instructions Loading

When your request matches a skill's description, Copilot loads the full SKILL.md body into its context. Detailed instructions become available only at this point.

`Loads: full SKILL.md body
into context window`

3

Resource Access

Copilot can access additional files in the skill directory (scripts, examples, docs) only as needed. Resources don't load until referenced — keeping context efficient.

`Accesses: scripts, examples,
templates on demand`

On-demand loading keeps your context window efficient — only relevant skills consume tokens.

Agent Skills vs Custom Instructions

When to use each — and how they complement each other

Feature	Agent Skills	Custom Instructions
Purpose	Specialized capabilities & workflows	Coding standards & guidelines
Portability	VS Code, CLI, Coding Agent	VS Code and GitHub.com only
Content	Instructions, scripts, examples, resources	Instructions only (markdown)
Scope	Task-specific, on-demand loading	Always applied or via glob patterns
Standard	Open standard (agentskills.io)	VS Code / GitHub-specific
Loading	Loaded only when relevant	Always in context window



Use together: Custom instructions for always-on project standards (naming, formatting, architecture). Agent Skills for specialized workflows (testing, deployment, security scanning) that load on-demand.

Custom Agents in GitHub Copilot

Building Specialized AI Team Members Inside Your IDE



Creating a specialized team member inside Copilot — one for planning, one for coding, one for reviewing — and switching between them with a single click.



Role-Specific Personas



.agent.md Files



Multi-Agent Workflows

What Are Custom Agents?

Configure Copilot to adopt a specific persona for any development role



Custom agents let you configure Copilot to adopt a specific persona — like Security Reviewer, Solution Architect, Planner, or Code Reviewer.



Instructions

What to do, what to avoid — written in plain Markdown

Tools

What it can access or modify — scoped per role



Model Preference

Which LLM to use, with fallback options

Think of it as: Creating a specialized team member inside Copilot — and switching between them with a single click.

Security Reviewer

Solution Architect

Planner

Code Reviewer

Why Custom Agents?

The problem they solve — and the workflow they enable

✗ Without Custom Agents

- ⚠ Manually select tools every session
- ⚠ Re-paste instructions repeatedly
- ⚠ Switch context mentally between roles
- ⚠ Risk accidental operations (editing during planning)
- ⚠ Inconsistent behavior across sessions

✓ With Custom Agents

- ✓ One-click role switching
- ✓ Pre-configured tools per role
- ✓ Consistent behavior every time
- ✓ Tool scoping prevents accidental operations
- ✓ Guided handoff workflows between agents

Example: A "Plan" agent uses only read-only tools (fetch, search) — it literally cannot modify code. No accidental edits during planning sessions.

The .agent.md File

The building block: a Markdown file with YAML frontmatter + instructions

File Structure

```
---
description: Generate plans
tools: ['fetch', 'search']
model: ['Claude Opus 4.5']
handoffs:
  - label: Implement Plan
    agent: agent
---

# Instructions (Markdown)
You are a planning agent...
```



Storage Locations

`.github/agents/`

Workspace-level
(shared with team)

User Profile

Personal, reusable
across all workspaces

`.claude/agents/`

Claude Code
compatible format

`planner.agent.md` → "Planner" in dropdown

Structure: YAML Frontmatter (configuration: tools, model, handoffs) + Body (instructions in plain Markdown)

Real Example: Planning Agent

A read-only agent that generates implementation plans without modifying code

planner.agent.md

```
---
description: Generate an implementation
  plan for new features
name: Planner
tools:
  ['fetch', 'githubRepo', 'search', 'usages']
model:
  ['Claude Sonnet 4.6', 'GPT-5.2-Codex']
handoffs:
  - label: Implement Plan
    agent: agent
    send: false
---
# Planning Instructions
You are in planning mode.
Do NOT make any code edits.
```



Read-Only Tools

No editFiles — literally cannot modify code during planning



Model Fallback

Tries Claude Opus 4.7 first, falls back to GPT-5



Handoff Button

"Implement Plan" button hands off to coding agent



send: false

User previews the prompt before it fires to next agent

Handoffs: The Workflow Game Changer

Chain agents together in guided workflows — without losing context



send: false

User previews the prompt
before it fires

send: true

Automatically submits and starts
next agent

Use Case — TDD Workflow: "Write Failing Tests" agent → "Make Tests Pass" agent. Each step reviewed before proceeding.

Key Frontmatter Fields

Quick reference for .agent.md configuration options

Field	Purpose	Example
name	Agent display name	"Security Reviewer"
description	Shown as placeholder text	"Review code for vulns"
tools	What the agent can use	['codebase', 'search']
model	LLM to use (with fallback)	['Claude Opus 4.5', 'GPT-5.2']
handoffs	Next-step suggestions	label, agent, prompt, send
user-invokable	Show in dropdown?	false (for subagents only)
agents	Which subagents can be used	['Researcher', 'Implementer']

Tool Priority & Best Practices

Design principles for safe, effective custom agents

Tool Priority Order (Highest → Lowest)

- ① Tools in the Prompt File (highest)
- ② Tools from referenced custom agent
- ③ Default tools of the selected agent

Design Principles

- ✓ **Least Privilege** — Only give agents the tools they need
- ✓ **Single Responsibility** — One agent, one job
- ✓ **Use Handoffs** — Guide workflow, don't automate blindly

Agent	Role	Allowed Tools	Restricted From
Planner	Architecture planning	fetch, search, githubRepo	editFiles
Coder	Implementation	editFiles, terminal	—
Reviewer	Code review	codebase, search	editFiles
Security Auditor	Vulnerability scan	codebase, fetch	editFiles, terminal
TDD Writer	Write failing tests	editFiles	terminal (run)

Custom Agents Key Takeaways

- Custom agents = reusable, role-specific Copilot configurations stored as .agent.md files
- Each agent has scoped instructions, tools, and model preferences — enforcing least privilege by design
- Handoffs enable guided, multi-step developer workflows: Plan → Code → Review → Ship
- Subagents enable orchestrated, autonomous multi-agent pipelines with narrowly scoped toolsets
- Three sharing levels: workspace (.github/agents/), organization-wide, and Claude Code compatible
- Tool priority: Prompt File > Referenced Agent > Default Agent — giving you fine-grained control
- Available in VS Code 1.106 and later — expanding to other IDEs



Awesome Copilot

<https://github.com/github/awesome-copilot/tree/main>